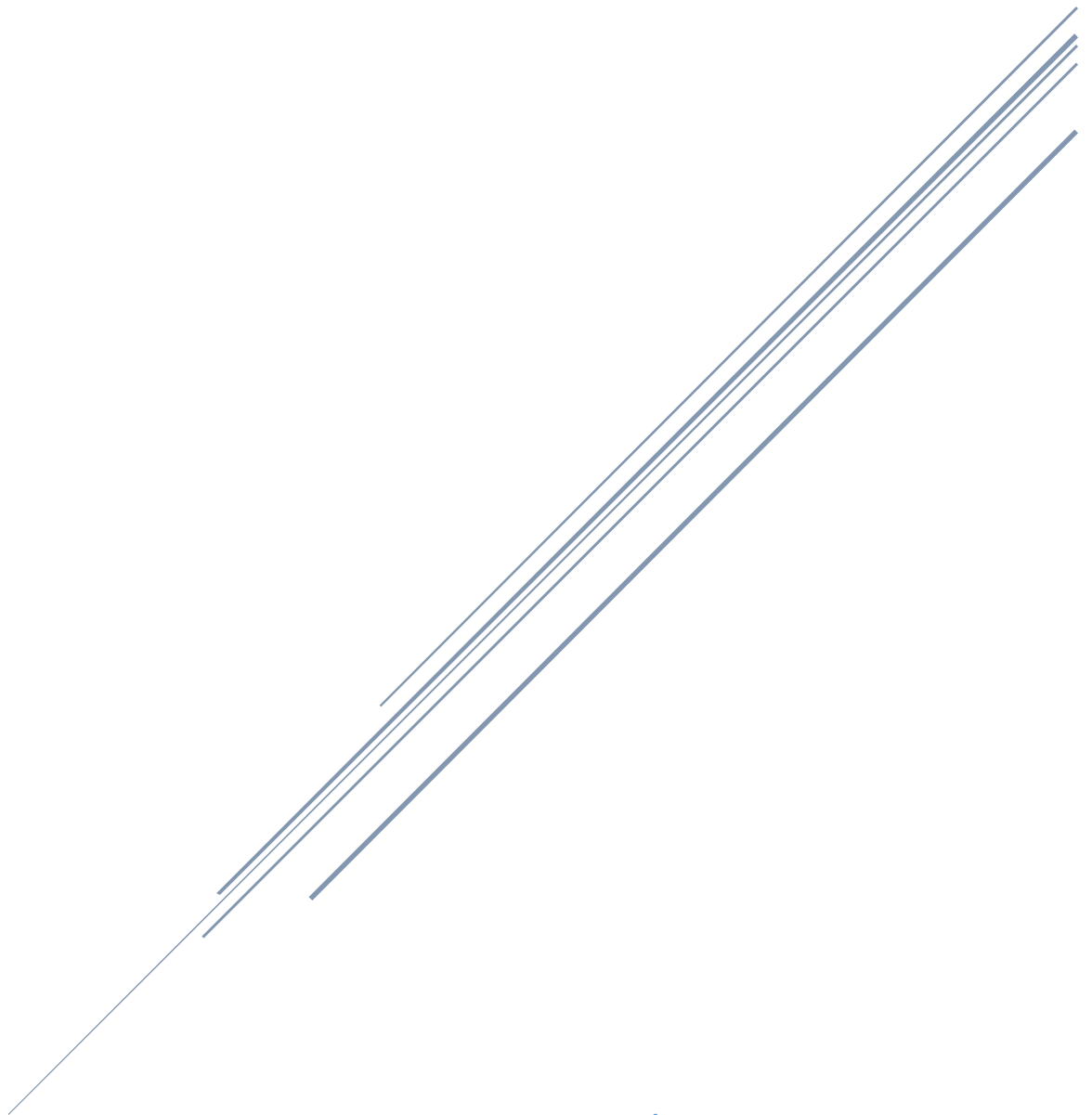


CODE AND SUMMARY OF CLASS 10

Sahir Ahmed Sheikh

Saturday (2 – 5)



Teachers:

Muhammad Bilal And Ali Aftab Sheikh

Code And Summary Of Class 10 – Saturday (2 – 5) | Quarter 3

Introduction

This document provides a comprehensive summary of Class 10 of Quarter 3, held on Saturday from 2-5 PM, led by Sir Ali Aftab Sheikh and Sir Muhammad Bilal. As the second-to-last class of Quarter 3, the session focused on completing the revision of Object Oriented Programming (OOP) concepts, with a deep dive into the fourth pillar, **Abstraction**, alongside advanced topics like **constructors**, **destructors**, **composition**, **aggregation**, **Method Resolution Order (MRO)**, and **the __new__ method**. The class combined theoretical explanations, real-world analogies, coding demonstrations in Google Colab, and motivational advice to prepare students for the upcoming viva and Quarter 4 challenges.

The class addressed the following key objectives:

- Completing the OOP pillars with a focus on Abstraction and its implementation.
- Introducing advanced OOP concepts like composition, aggregation, and MRO.
- Clarifying the role of constructors, destructors, and the __new__ method.
- Preparing students for the viva scheduled for the next (final) class, covering note books 12 and 13.
- Emphasizing self-learning, active participation, and industry-relevant skills.

Students were reminded that Quarter 3 has only one class remaining, with a viva scheduled for May 24, 2025, covering notebooks 12 and 13. The viva will include theoretical questions and coding problems, requiring students to write and explain code. Quarter 4, starting in June, was highlighted as significantly more challenging, with topics like AI Agents and cloud computing.

Topics Covered

1. Class Updates and Quarter 3 Context

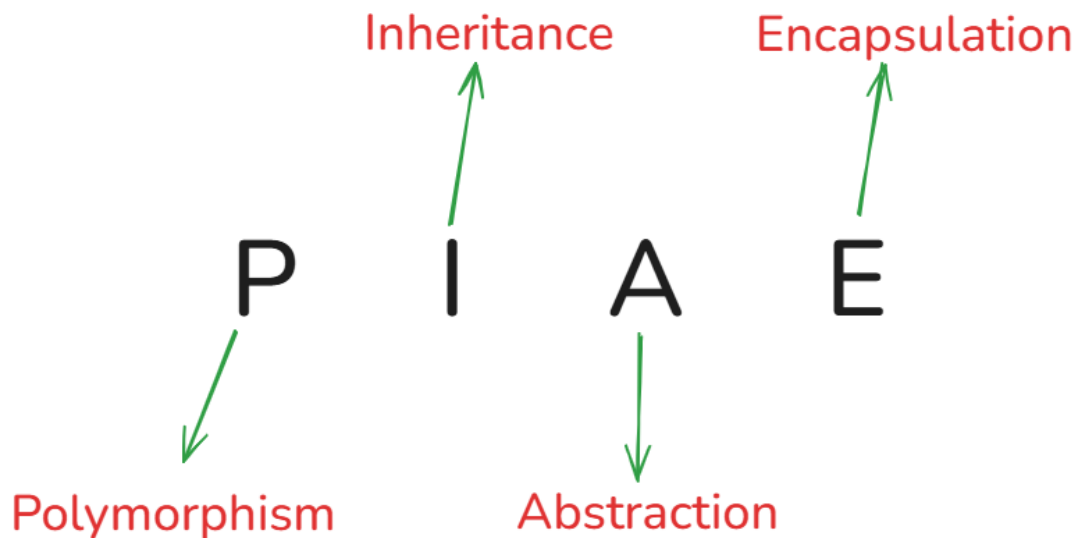
- **Quarter 3 Overview:** With only one class left, Quarter 3 was described as relatively relaxed, but students were warned that Quarter 4 will introduce advanced topics like AI and cloud computing, requiring immediate preparation. The syllabus is designed to align with industry demands, offering high salary potential.
- **Viva Announcement:** The viva, scheduled for May 31, 2025, will cover notebooks 12 and 13, including theoretical questions and coding problems. Students may be called on stage to solve problems, with the viva lasting approximately two hours. Failure to attend or perform well may lead to course discontinuation
- **CR Announcement:** Two new CR assistants were announced, bringing the total to three male CR assistants and five female CRs. Eight students were selected for the Quarter 4 team, with roles to be assigned based on assignment submissions.
- **Motivational Advice:** Sir Ali shared his journey from commerce to IT, emphasizing resilience, self-learning, and the importance of effort over immediate results. Students were urged to avoid demotivation, build LinkedIn profiles, and showcase skills to secure internships (starting at 30-35k PKR).

*2. Revision of OOP Concepts**:*

The session revisited the four pillars of OOP (Polymorphism, Inheritance, Abstraction, Encapsulation) using the PIAE mnemonic. The class-object relationship was reinforced with the blueprint analogy:

- **Class:** A blueprint defining attributes and methods, e.g., a bank's template for accounts.
- **Object:** An instance of a class, e.g., a specific bank account with unique values.

3. Four Pillars of OOP (PIAE Mnemonic)



The class completed the discussion of the four pillars, focusing heavily on Abstraction, while briefly reviewing Polymorphism, Inheritance, and Encapsulation.

3.1 Polymorphism

- **Definition:** Polymorphism allows objects of different classes to share a common interface, with unique behaviors via method overriding.
- **Real-World Analogy:** A remote control's "play" button plays a movie on a DVD player and a song on a music player, demonstrating the same action with different outcomes.
- **Types:**
 - **Compile-time (Static):** Method overloading (less common in Python).
 - **Run-time (Dynamic):** Method overriding (common in Python via inheritance).
- **Benefits:** Simplifies code, enhances reusability, and supports scalability by allowing a single interface for varied behaviors.

- **Coding Example:**

```
class BankAccount:
    def send_notification(self):
        return f"Hello {self.account_holder}, your bank balance is {self._balance}"

class StudentAccount(BankAccount):
    def send_notification(self): # Override parent method
        return f"Hello {self.account_holder}, your pocket money is {self._balance}"

# Create objects
account = BankAccount("Ali", 1000)
student = StudentAccount("Ahmed", 500)

# Demonstrate polymorphism
print(account.send_notification()) # Output: Hello Ali, your bank balance is 1000
print(student.send_notification()) # Output: Hello Ahmed, your pocket money is 500
```

- **Explanation:** The `send_notification` method is overridden in `StudentAccount` to provide a different message, showcasing polymorphism. The same method name produces different outputs based on the object type.

3.2 Inheritance

- **Definition:** Inheritance enables a child class to reuse a parent class's attributes and methods, supporting "is-a" relationships.
- **Real-World Analogy:** A family business passing down practices to the next generation.
- **Types:**
 - Single Inheritance: One parent, one child.
 - Multiple Inheritance: One child, multiple parents.
 - Multilevel Inheritance: Grandparent → Parent → Child.
 - Hierarchical Inheritance: One parent, multiple children.
- **Benefits:** Reduces code duplication, supports extensibility, and models real-world hierarchies.
- **Coding Example:**

```

class BankAccount:
    def __init__(self, account_holder, balance):
        self.account_holder = account_holder
        self._balance = balance

    def deposit(self, amount):
        if amount > 0:
            self._balance += amount
            return f"Deposited {amount}. New balance: {self._balance}"
        return "Invalid deposit amount"

class SavingsAccount(BankAccount):
    def __init__(self, account_holder, balance, interest_rate):
        super().__init__(account_holder, balance) # Inherit parent's constructor
        self.interest_rate = interest_rate

    def add_interest(self):
        interest = self._balance * self.interest_rate
        self._balance += interest
        return f"Added interest {interest}. New balance: {self._balance}"

# Create objects
savings = SavingsAccount("Ali", 1000, 0.05)
print(savings.deposit(500)) # Output: Deposited 500. New balance: 1500
print(savings.add_interest()) # Output: Added interest 75.0. New balance: 1575

```

- **Explanation:** `SavingsAccount` inherits `deposit` from `BankAccount` and adds `add_interest`, demonstrating code reuse and extension. The `super()` function calls the parent's constructor to initialize inherited attributes.

3.3 Abstraction

- **Definition:** Abstraction hides complex implementation details, exposing only necessary interfaces using abstract base classes (ABCs) via the `abc` module.
- **Real-World Analogy:** Using an ATM's deposit/withdraw buttons without knowing the bank's backend processes. Similarly, serving biryani focuses on the dish, not the recipe's complexity.
- **Benefits:** Simplifies user interaction, enforces method implementation in child classes, and reduces errors.
- **Coding Example:**

```

from abc import ABC, abstractmethod

class Bank(ABC):
    @abstractmethod
    def withdraw(self, amount):
        pass

    @abstractmethod
    def deposit(self, amount):
        pass

    @abstractmethod
    def loan(self, amount):
        pass

class SavingsAccount(Bank):
    def __init__(self, account_holder, balance):
        self.account_holder = account_holder
        self.__balance = balance

    def withdraw(self, amount):
        if amount <= self.__balance:
            self.__balance -= amount
            return f"Withdrew {amount}. New balance: {self.__balance}"
        return "Insufficient funds"

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            return f"Deposited {amount}. New balance: {self.__balance}"
        return "Invalid deposit amount"

    def loan(self, amount):
        return f"Loan of {amount} not available for savings account"

# Create instance
s1 = SavingsAccount("Ali", 1000)

print(s1.deposit(500)) # Output: Deposited 500. New balance: 1500
print(s1.withdraw(200)) # Output: Withdrew 200. New balance: 1300
print(s1.loan(1000)) # Output: Loan of 1000 not available for savings account

```

- Explanation:** Bank is an abstract class with abstract methods withdraw, deposit, and loan. SavingsAccount inherits Bank and implements all abstract methods, ensuring a consistent interface. Attempting to instantiate Bank directly (`b1 = Bank()`) raises an error, as abstract classes cannot be instantiated. The loan method was included as an abstract method to demonstrate mandatory implementation, but SavingsAccount restricts loans, showing flexibility in logic.

3.4 Encapsulation

- **Definition:** Encapsulation restricts direct access to attributes, using public, protected (`_attribute`), and private (`__attribute`) modifiers.
- **Real-World Analogy:** A bank account hiding balance details but allowing transactions via methods.
- **Python's Approach:**
 - **Public:** Accessible everywhere (e.g., `account_holder`).
 - **Protected:** Indicated by `_attribute` (convention, not enforced).
 - **Private:** Indicated by `__attribute` (name mangling, restricts access).
- **Benefits:** Ensures data integrity, supports modularity, and allows controlled access with validation.
- **Coding Example:**

```
class BankAccount:
    def __init__(self, account_holder, initial_balance):
        self.account_holder = account_holder # Public
        self.__balance = initial_balance # Private

    def get_balance(self):
        return self.__balance

    def deposit(self, amount):
        if amount > 0:
            self.__balance += amount
            return f"Deposited {amount}. New balance: {self.__balance}"
        return "Invalid deposit amount"

    def withdraw(self, amount):
        if 0 < amount <= self.__balance:
            self.__balance -= amount
            return f"Withdrew {amount}. New balance: {self.__balance}"
        return "Invalid or insufficient funds"

# Create object
account = BankAccount("Ali", 1000)
print(account.account_holder) # Output: Ali
print(account.get_balance()) # Output: 1000
print(account.deposit(500)) # Output: Deposited 500. New balance: 1500
print(account.withdraw(200)) # Output: Withdrew 200. New balance: 1300
print(account.withdraw(2000)) # Output: Invalid or insufficient funds
# print(account.__balance) # AttributeError: No attribute __balance
```

- **Explanation:** The `__balance` attribute is private, accessible only through `get_balance`, `deposit`, and `withdraw`. Validation in methods ensures safe modifications, and direct access to `__balance` raises an error, demonstrating data protection.

4. Advanced OOP Concepts

4.1 Constructors and Destructors

- **Constructor (__init__):** Initializes an object's attributes when an instance is created. It sets values in the self dictionary, which represents the object.
- **Destructor (__del__):** Runs when an instance is deleted, allowing cleanup tasks (e.g., closing database connections).
- **Real-World Analogy:** A car factory (constructor) assembles a car, while a scrapyard (destructor) dismantles it.
- **Coding Example:**

```
class Car:
    def __init__(self, brand, model):
        self.brand = brand
        self.model = model
        print(f"{self.brand} {self.model} has been created")

    def __del__(self):
        print(f"{self.brand} {self.model} has been destroyed")

# Create and delete instance
c1 = Car("Toyota", "Corolla")
del c1  # Output: Toyota Corolla has been created
        #          Toyota Corolla has been destroyed
```

- **Explanation:** The constructor runs on instance creation, printing a creation message. The destructor runs when `del c1` is called, printing a destruction message. The `dir(c1)` function was used to inspect object attributes, revealing magic methods like `__init__` and `__del__`.

4.2 Composition and Aggregation

- **Composition:** A “has-a” relationship where one class owns another, and the dependent class's instance is destroyed when the parent is. E.g., a car owns an engine; if the car is destroyed, the engine is too.

- **Aggregation:** A “has-a” relationship where the dependent class can exist independently. E.g., a university has departments, but departments can exist without the university (e.g., as coaching centers).
- **Coding Example:**

```
class Engine:
    def start(self):
        return "Engine started"

class Car:
    def __init__(self):
        self.engine = Engine() # Composition: Car owns Engine

    def start(self):
        return self.engine.start()

class Department:
    def __init__(self, name):
        self.name = name

class University:
    def __init__(self):
        self.departments = []

    def add_department(self, dept):
        self.departments.append(dept) # Aggregation: University has Department

# Composition
c1 = Car()
print(c1.start()) # Output: Engine started

# Aggregation
u1 = University()
d1 = Department("Computer Science")
u1.add_department(d1)
```

- **Explanation:** In composition, Car creates an Engine instance, which is destroyed when Car is. In aggregation, University references Department instances, which can exist independently.

4.2 Method Resolution Order (MRO)

- **Definition:** MRO determines the order in which Python searches for methods in a class hierarchy, following the C3 linearization algorithm.
- **Real-World Analogy:** Tracing a family lineage from a child to ancestors (e.g., from you to Hazrat Adam A.S.).
- **Coding Example:**

```
class Car:
    pass

print(Car.mro()) # Output: [<class '__main__.Car'>, <class 'object'>]
```

- **Explanation:** Car inherits from object by default. mro() returns a list starting with Car, followed by object, showing the method lookup order. MRO ensures super() calls the correct parent class method.

4.2 The __new__ Method

- **Definition:** __new__ is a static method that creates an object and allocates memory before __init__ initializes it. It returns the object to self in __init__.
- **Real-World Analogy:** __new__ builds the car's frame, while __init__ adds the engine and paint.
- **Key Insight:** __new__ manages memory allocation in the heap, while __init__ sets attributes. Python automatically calls __new__ before __init__.
- **Coding Example:**

```
class Car:
    def __new__(cls, *args, **kwargs):
        instance = super().__new__(cls) # Allocate memory
        return instance

    def __init__(self, brand):
        self.brand = brand
```

- **Explanation:** `__new__` creates an empty object, stored in the heap, and returns it to `__init__`. The heap stores object values, while the stack holds references (e.g., variable names).

5. Practical Coding and Debugging

- **Google Colab:** Used for coding exercises, with students encouraged to practice abstraction and constructor/destructor implementations.
- **Debugging Example:** Addressed errors like missing abstract method implementations (e.g., `loan` in `SavingsAccount`) and missing constructor arguments. Students were guided to use `pass` or default values to avoid errors temporarily.
- **Validation Logic:** Added conditions to `withdraw` and `deposit` methods to ensure valid transactions, reinforcing robust coding practices.

6. Key Insights and Takeaways

- **OOP's Power:** The four pillars, combined with advanced concepts like composition and MRO, enable modular, scalable code, preparing students for industry projects.
- **Self-Learning:** Industry success requires independent learning, as classes alone are insufficient. Students must explore notebooks 12 and 13 independently.
- **Viva Preparation:** The viva is critical, testing both theory and coding. Missing it risks failure and exclusion from Quarter 4.
- **Industry Relevance:** Quarter 4's syllabus (e.g., cloud computing, AI) aligns with high-demand skills, offering salaries of 10+ lakhs PKR for experienced professionals.
- **Resilience:** Sir Ali's story highlighted that mistakes (e.g., choosing commerce) are learning opportunities. Consistent effort and portfolio-building (e.g., LinkedIn, GitHub) are key.

Class Resources & Assignments

Today's Class Code

- The code for Class 10 is available on Google Colab: [Class Code](#)
- Students are encouraged to experiment with the provided examples (e.g., implementing abstract methods, creating/destroying objects) to solidify understanding.

Assignments

1. **Complete Previous Assignments:**
 - Ensure all prior assignments are completed, shared on LinkedIn, and submitted via the class submission form.
 - Review feedback to improve code quality and understanding.
 - **Deadline:** Before the next class.

Important Reminder

- The next class (May 24, 2025) is the final class of Quarter 3, including the viva. Attendance is mandatory, as missing the viva results in automatic failure and exclusion from Quarter 4. Students with valid reasons (e.g., board exams) must inform management.
- Complete all assignments to secure roles in the Quarter 4 team. The CR and assistants will review submissions and assign responsibilities.
- Use Sir Arif's slides and notebooks 12 and 13 for viva preparation, as they form the basis of theoretical and coding questions.
- Stay consistent, ask questions, and engage actively to master OOP, which is foundational for Quarter 4 and industry success.

Final Words

The responsibility is yours. Actively learn, confidently participate, and practice daily. The viva is a critical milestone, and Quarter 4 will demand greater effort with advanced topics like AI and cloud computing. Embrace challenges, complete assignments, and build your portfolio to seize IT industry opportunities. Stay focused, and may Allah guide you.

Allah Hafiz – See you in the next class!

Repo Link: [GIAIC-Sat-Afternoon](#)

This is my repository where I have uploaded all the class codes, assignments, and detailed summaries. Review each class's code and summary to prepare for the viva and Quarter 4

Thank You for Reading!

Hope you understood Class 10 well.

"Education is not preparation for life; education is life itself." – *John Dewey*